S305 — OBJECT ORIENTED PROGRAMMING & METHODOLOGY

Complete Course Notes with Java Implementation

Definitions + UML-style Diagrams + Editor-style coloured Java code + Real-World Applications | 5 Units

SYLLABUS COVERED

Unit 1 OO thinking vs procedural, features (merits/demerits) of OO, object model, elements of OOPS, IO processing	**Unit 2** Encapsulation & abstraction, object (state/behaviour/identity), classes & candidates, attributes/services, access modifiers, statics, instances, message passing, construction/destruction	**Unit 3** Inheritance purpose & types, is-a, association, aggregation, composition, abstract classes, interfaces	**Unit 4** Polymorphism, method overloading & overriding, static vs run-time polymorphism
Unit 5 Strings, exception handling, multi-threading, collections; Case studies: ATM, Library Management System	**Style** Editor-style syntax-highlighted Java + colourful layouts + comparison tables + exercises	**Language note** All examples Java (JDK 8+ syntax). Pointer/memory C topics marked where the syllabus pair uses C++	**How to use** Read units in order; compile every box: javac File.java; java File

Prepared as a self-study & exam-prep guide following the S305 syllabus. Code follows modern Java (try-with-resources, lambdas, streams) while the theory is language-neutral.

TABLE OF CONTENTS

Study tip: implement each coloured code box in your editor (JDK + any IDE), run it, then redraw the diagrams on paper. Finish every unit's Exercise line.

UNIT 1 | OO THINKING, FEATURES & OBJECT MODEL

1.1 Procedural Programming vs Object Oriented Thinking

WHAT IS PROGRAMMING PARADIGM?

A paradigm is a style / way of thinking in which a program is written. Two dominant ones: PROCEDURAL / STRUCTURED (C, Fortran, Pascal) where we think in terms of FUNCTIONS acting on data, and OBJECT ORIENTED (Java, C++, Python, C#) where we think in terms of OBJECTS — data bundled with the operations that act on it.

```
PROCEDURAL view of a Bank:                  OO view of a Bank:
  data[30]  (accounts[] array)             object: AccountKeeper
  function deposit(a, amt)                 view:  + openAcc()
  function withdraw(a, amt)                view:  + deposit()
  function print(a)                        view:  + withdraw()
  --- data and functions are SEPARATE ---  --- data + methods in one capsule ---
  risk: any function may corrupt data      data is private; only accepted messages
```

Basis	Procedural (C)	Object Oriented (Java)
Unit of code	Functions / procedures	Classes & objects
Focus	"What has to be done" (process)	"Who does the job" (entities)
Data security	Weak - global data exposed	Strong - data is private (encapsulated)
Approach	Top-down (divide into functions)	Bottom-up (model real objects first)
Code reuse	Function libraries	Inheritance, composition, generics
Extensions	Hard - change ripples	Easy - add new classes
Testing	Isolated functions	Objects; easier mocking in real systems
Examples	C, COBOL, Fortran	Java, C#, C++, Python, Kotlin

1.2 Features of the Object Oriented Paradigm

Encapsulation Data + methods packed in one unit; data hidden from outside.	**Abstraction** Show only essential behaviour, hide complexity (an ATM screen).	**Inheritance** A child class acquires members of a parent class (is-a).	**Polymorphism** Same method name behaves differently per object (many forms).
Message passing Objects communicate by calling each other's methods.	**Modularity** Program = independent, exchangeable parts (classes).	**Dynamic binding** Decide which method to run at run time.	**Reusability** Classes are reusable via inheritance & composition.

MERITS (ADVANTAGES) OF OO METHODOLOGY

Modular & maintainable code, data hiding (security), easier modelling of real-world problems, reuse (inheritance + libraries), polymorphism gives flexible design, big-team & big-project friendly, well suited for GUI/network/game systems.

DEMERITS (DISADVANTAGES)

Steeper learning curve, larger programs & slower start, more design effort (classes/relationships needed up-front), sometimes over-abstraction, more memory (object headers, virtual tables), not best for pure number-crunching / device drivers where procedural C is lean.

KEY POINT

Which is better? Neither is "best" — Unix kernel (C), OS schedulers and compilers are procedural; enterprise apps (banking, e-commerce, Android) are OO. Modern languages mix both: Java services are OO, but inside methods you write procedural steps.

1.3 Object Model

OBJECT MODEL (BOOCH — FOUR MAJOR ELEMENTS)

A collection of concepts forming a foundation: 1) ABSTRACTION — simplify reality into essentials, 2) ENCAPSULATION — hide implementation, 3) MODULARITY — break into modules, 4) HIERARCHY — organise with inheritance/composition.

OBJECT MODEL — SUPPORTING (MINOR) ELEMENTS

TYPING (safe, checked types), CONCURRENCY (objects running simultaneously as threads), PERSISTENCE (objects survive beyond program lifetime — databases, files)).

```
                O B J E C T   M O D E L
+-------------------------------------------------------------+
|  MAJOR elements                  MINOR elements             |
|  1. Abstraction                  5. Typing (static/dynamic) |
|  2. Encapsulation                6. Concurrency (threads)   |
|  3. Modularity                   7. Persistence (file/DB)   |
|  4. Hierarchy (inheritance +      |                         |
|      aggregation)                 +-------------------------+
+-------------------------------------------------------------+
Example mapping:  Customer class (abstraction), private pin (encap-
sulation), accounts package (modularity), CurrentAccount extends
Account (hierarchy), Account typed var (typing), 2 login threads
(concurrency), data saved to MySQL (persistence).
```

1.4 Elements of OOPS

Object Instance of a class, has state + behaviour + identity.	**Class** Blueprint/template describing a group of objects.	**Method** Behaviour (service) an object can perform.	**Attribute (field)** Data stored by an object (its state).
Message A request object-A sends to object-B (a method call).	**Interface** Contract of behaviour (list of method signatures).	**Constructor** Special method that builds/initialises an object.	**Package / Module** Grouping unit for related classes.

```
CLASS "Student" (blueprint)              THREE OBJECTS (instances)
-------------------------------------------------------------
attributes: rollNo, name, marks          +--------+  +--------+  +--------+
methods   : display(), calcGrade()       | 101    |  | 102    |  | 103    |
(one definition)                         | Aisha  |  | Rohan  |  | Meera  |
                                        | 89     |  | 71     |  | 94     |
                                        +--------+  +--------+  +--------+
Each object reserves its OWN memory for attributes but shares the
same class methods code (methods live once in the class).
```

1.5 IO Processing in Java

STREAMS

Java treats input/output as a STREAM — a flow of bytes/chars from a source to a destination. System.in (keyboard), System.out (console), files, network sockets are all streams. Character streams (Reader/Writer) handle text; byte streams (InputStream/OutputStream) handle binary.

IODemo.java

```java
import java.util.Scanner; // modern console input
public class IODemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in); // reads from keyboard
        System.out.print("Enter name: ");
        String name = sc.nextLine(); // one whole line
        System.out.print("Enter age: ");
        int age = sc.nextInt(); // parse an int
        sc.close(); // release the resource
        System.out.printf("%s is %d years old.%n", name, age);
        System.err.println("(errors go to System.err)"); // for logs
    }
}
```

FileIODemo.java

```java
import java.io.*;
public class FileIODemo {
    public static void main(String[] args) throws IOException {
        // WRITE characters to a file
        try (BufferedWriter w = new BufferedWriter(
                new FileWriter("notes.txt"))) {
            w.write("Hello OOP class!"); // auto-flushed & closed
            w.newLine();
            w.write("Second line");
        } // try-with-resources auto-closes
        // READ them back
        try (BufferedReader r = new BufferedReader(
                new FileReader("notes.txt"))) {
            String line;
            while ((line = r.readLine()) != null)
                System.out.println("READ: " + line);
        }
    }
}
```

KEY POINT

Real world: every Spring Boot web request reads System.in-like streams over HTTP sockets; every logger (Log4j/SLF4J) writes to System.err / files. Buffered streams exist because a system call per char is slow — a buffer batches many chars into few calls.

1.6 Thinking in Objects — a first complete walkthrough

THE OO DESIGN PROCESS IN 4 STEPS

(1) READ the problem, (2) NOUNs → candidate classes, (3) VERBs → methods, (4) draw the object interaction (message passing). Example: a parking ticket machine.

ParkingMachine.java

```java
import java.util.Scanner;
public class ParkingMachine {
    private int ratePerHr; // state (encapsulated)
    public ParkingMachine(int rate){ ratePerHr = rate; } // constructor
    public double computeFee(int vehicleType, double hours){ // service / behaviour
        double multiplier = (vehicleType == 1) ? 1.0 : 0.5; // car vs bike
        return Math.round(ratePerHr * hours * multiplier);
    }
    public static void main(String[] a){
        ParkingMachine m = new ParkingMachine(20);
        Scanner sc = new Scanner(System.in);
        System.out.print("Car(1) or Bike(2)? ");
        int type = sc.nextInt();
        System.out.print("Hours parked? ");
        double hrs = sc.nextDouble();
        double fee = m.computeFee(type, hrs); // <- MESSAGE to the machine
        System.out.printf("Please pay Rs %.2f %n", fee);
    }
}
```

```
Classes vs objects discovered from the wording:
"The machine charges a car 20/hr and a bike 10/hr by hours parked."
Noun classes : ParkingMachine, (Ticket)     |  attributes: ratePerHr
Verb methods : computeFee(), printTicket()  |  identity  : each machine obj
Message flow : main -----computeFee(type,hrs)----> machine
```

One class	One interface	One method	One package
Define attributes + methods once; new -> many objects, each with own data	Contract of behaviour implemented by many classes: one call, many answers.	Logic written once, invoked from many places = reuse + central fix.	Related classes grouped, imported by many apps = clean modular builds.

Real-world snippet	Procedural-ish way	OO way (message)
E-commerce cart	total = sum(cart.items)	cart.calculateTotal()
Bank transfer	transfer(from, to, amt)	sender.transferTo(receiver, amt)

Real-world snippet	Procedural-ish way	OO way (message)
Music player	playSong(song)	player.play(song)
WhatsApp	send(contact, text)	chat.sendMessage(text)

KEY TERMS GLOSSARY

Interface: what an object can do. Client: the code that sends messages. Server: the object that receives them. Constructor: initialising builder. Garbage: unreachable objects. Class variable: static field. Instance variable: non-static field.

Exercise: *1) Write C-style code for "deposit & withdraw" and its Java OO version; compare the data-flow. 2) Add a GUI/console menu bank with Scanner. 3) Write a program that copies a text file line-by-line and prints total lines (using BufferedReader).*

UNIT 2 | ENCAPSULATION, OBJECTS & CLASSES

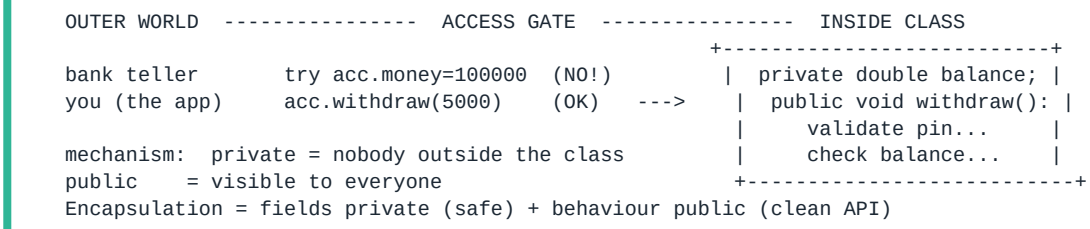
2.1 Encapsulation & Data Abstraction

ENCAPSULATION (INFORMATION HIDING)

Bundling data (attributes) and the methods that operate on that data into one unit (a class), and RESTRICTING direct access to the data from outside. Outside code can only touch the data through public methods. In Java: private fields + public getters/setters.

ABSTRACTION

Show the ESSENTIAL interface, hide the complex implementation. A driver only sees accelerate(), brake(), steer() — not the engine internals. In Java: abstract classes, interfaces, and public APIs.



BankAccount.java

```java
public class BankAccount {
    private double balance; // DATA IS PRIVATE (no: acc.balance)
    public BankAccount(double init) { balance = init; } // constructor
    public void deposit(double amt) { // public SERVICE
        if (amt > 0) balance += amt;
    }
    public double getBalance() { return balance; } // read-only view
}
// from outside:
// acc.deposit(500); System.out.println(acc.getBalance());
// acc.balance = 999 -> COMPILE ERROR (encapsulation enforced)
```

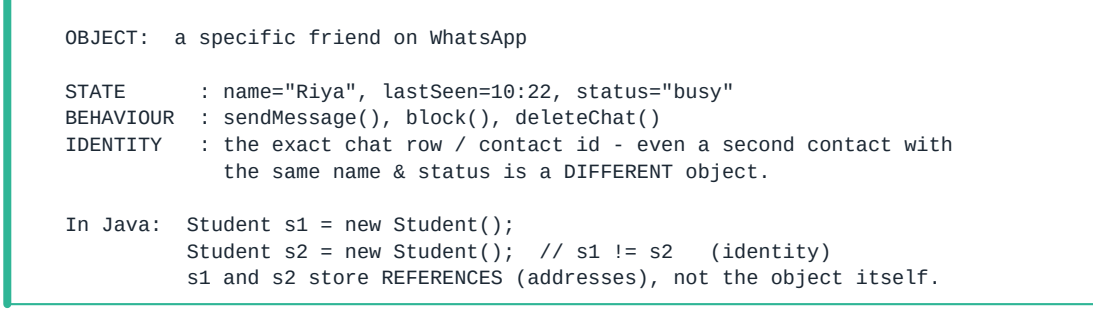
KEY POINT

Why private fields + getters/setters? Because later you can add validation, logging or convert to a database without breaking every caller. That flexibility is worth the 3 extra lines. Real example: every ORM layer (Hibernate) reads/writes private fields through reflection — safe BECAUSE the class controls its own contract.

2.2 Concept of Objects — State, Behaviour, Identity

DEFINITION: OBJECT

An object is a runtime entity that has: 1) STATE — values of its attributes at a moment (e.g., balance = 5000), 2) BEHAVIOUR — the methods it can execute (deposit, withdraw), 3) IDENTITY — a unique existence distinguishable from other objects even with same state (obj1 != obj2 even if balance equal).



Student.java

```java
public class Student {
    // STATE (attributes)
    private int roll; private String name;
    // BEHAVIOUR (methods)
    public void set(int r, String n){ roll = r; name = n; }
    public void display(){ System.out.println(roll+" "+name); }
    public static void main(String[] a){
        Student s1 = new Student(); // s1 ---> objectA (identity#1)
        Student s2 = new Student(); // s2 ---> objectB (identity#2)
        s1.set(1, "Aisha"); s2.set(1, "Aisha");
        System.out.println(s1 == s2); // false (default == compares refs)
        System.out.println(s1.equals(s2)); // false unless you OVERRIDE equals()
    }
}
```

2.3 Classes — recognising them & candidate classes

DEFINITION: CLASS

A class is a BLUEPRINT/template that defines attributes and methods. Objects are created from the class at run time. One class, many objects.

HOW TO FIND CLASSES (ANALYSIS)

1) NOUN ANALYSIS: circle all nouns in the problem statement → candidate classes (Student, Account, Book, Loan). VERBS become methods (deposit, issueBook). 2) CRC CARDS (Class–Responsibility–Collaborator): one card per candidate; write responsibilities on left, collaborators (other classes it needs) on right. 3) Filter: drop nouns that are just attributes.

LibraryFind.java

```java
// Problem: "A library issues books to members and charges fine"
// Nouns -> candidates: Library, Book, Member, Issue, Fine, MemberCard
// Verbs -> methods: issue(), returnBook(), calculateFine()
// Attributes: some nouns (title, author) are attributes of Book, not classes.
public class Book {
    private String title, author; // attributes of Book
    public Book(String t, String a){ title=t; author=a; }
    public String getTitle(){ return title; }
}
public class Member {
    private String name;
    public void borrow(Book b){ ... } // collaboration: Member uses Book
}
```

Attributes & Services (verb analysis)

Class candidate	Attributes (Nouns - data)	Services (Verbs - behaviour)
Book	title, author, isbn, price	getTitle(), isAvailable(), issue()
Member	name, memberId, fineDue	borrow(), returnBook(), payFine()
Account	accNo, balance, ownerName	deposit(), withdraw(), getBalance()
ATM Card	cardNo, pin, expiry	authenticate(pin)

2.4 Access Modifiers & Static Members

ACCESS MODIFIERS

Keywords controlling visibility of a class member (field, method, constructor).In Java: private (only inside class), default/package-private (same package), protected (package + subclasses anywhere), public (everywhere).

Modifier	Same class	Same package	Subclass (any package)	World (anywhere)
private	YES	NO	NO	NO
(default) no keyword	YES	YES	NO	NO
protected	YES	YES	YES	NO

Modifier	Same class	Same package	Subclass (any package)	World (anywhere)
public	YES	YES	YES	YES

Static members — belonging to the CLASS, not to one object

WHAT STATIC MEANS

A static field is SHARED by all objects (one copy in memory). A static method is called using the ClassName and cannot touch instance fields directly. static blocks run once when the class loads. Used for: constants, counters, utility methods (Math.sqrt), main().

Counter.java

```
public class Counter {
    private int serial; // PER-OBJECT (instance field)
    private static int total = 0; // ONE shared copy across all objects
    public Counter(){ serial = ++total; } // each new object takes next id
    public static int getTotal(){ return total; } // static method (class level)
    public int getSerial(){ return serial; } // instance method (object level)
    public static void main(String[] a){
        Counter c1 = new Counter(); // serial 1
        Counter c2 = new Counter(); // serial 2
        System.out.println(Counter.getTotal()); // 2 (class-level access)
        System.out.println(c1.getSerial()); // 1 (object-level access)
        // Math.PI, Math.sqrt(...), Integer.MAX_VALUE are classic statics
        System.out.println(Math.max(3, 7)); // 7
    }
}
```

2.5 Instances & Message Passing

INSTANCE

An object created from a class (new ClassName(...) allocates memory and returns a reference). Each instance has its own copy of instance fields but shares class-level statics.

MESSAGE PASSING

Objects cooperate by sending MESSAGES to each other — in Java a message = invoking another object's public method (receiver.method(args)). The caller does not need to know HOW the receiver does the job.

MessagePassing.java

```
public class MessagePassing {
    static class Chef {
        public void cook(String dish){ // receives a message
            System.out.println("Chef cooked " + dish);
        }
    }
    static class Waiter {
        private Chef chef;
        public Waiter(Chef c){ chef = c; } // dependency injection
        public void takeOrder(String dish){
            chef.cook(dish); // <- MESSAGE to Chef object
        }
    }
    public static void main(String[] a){
        Waiter w = new Waiter(new Chef());
        w.takeOrder("pasta"); // Waiter -> Chef -> "cooked pasta"
    }
}
```

2.6 Construction & Destruction of Objects

CONSTRUCTOR

A special method with the SAME name as the class, no return type, called AUTOMATICALLY on new. Used to initialise state. Types: default (no-arg, compiler-made if you write none), parameterized, copy (build one object from another), and constructor chaining via this(...).

DESTRUCTION IN JAVA

Java has NO delete keyword and NO destructor. Objects die when unreachable (no references). The GARBAGE COLLECTOR (GC) reclaims their memory automatically. Options: finalize() (deprecated, unreliable), try-with-resources / close() for files and sockets, and System.gc() just a hint. In C++ the destructor ~ClassName() runs explicitly; in Java cleanup is deterministic only through closeable resources.

Car.java

```java
public class Car {
    private String model;
    private int speed;
    public Car() { this("Unknown", 0); } // constructor chaining
    public Car(String m, int s){ model = m; speed = s; } // parameterized
    public Car(Car other){ this(other.model, other.speed); } // copy constructor
    public void accelerate(){ speed += 10; }
    public String toString(){ return model + "@" + speed + "km/h"; }
    public static void main(String[] a){
        Car c1 = new Car("Swift", 40);
        Car c2 = new Car(c1); // copy
        c2.accelerate();
        System.out.println(c1 + " vs " + c2); // Swift@40 vs Swift@50
        Car tmp = c1; c1 = null; // object becomes unreachable
        // ... later the GC frees that memory automatically (no C++ delete)
    }
}
```

	C++ (for comparison)	Java
Creation	ClassName obj; or new with constructor	new ClassName(...)
Destruction	explicit destructor ~Class() runs on delete/scope	No delete; GC removes unreachable objects
Precision	deterministic	non-deterministic (GC decides when)
You must manage	memory manually (new/delete, copy ctor, =)	just stop referencing; close() for OS resources

2.7 Getters/Setters, this, equals & hashCode

GETTER / SETTER (ACCESSOR / MUTATOR)

The disciplined way to read/write a private field: getName() returns it, setName(v) validates then writes it. Guarantees invariants (age cannot become -5). this.x = x disambiguates the parameter from the field.

Person.java

```java
public class Person {
    private String name;
    private int age;
    public Person(String name, int age){ this.name = name; setAge(age); } // this.field
    public String getName(){ return name; } // getter
    public void setAge(int age){ if (age >= 0) this.age = age;
        else throw new IllegalArgumentException("age cannot be negative"); } // setter validates
    // identity !== equality: override equals + hashCode so same data = same value
    @Override public boolean equals(Object o){
        if (this == o) return true;
        if (!(o instanceof Person)) return false;
        Person p = (Person) o;
        return age == p.age && name.equals(p.name);
    }
    @Override public int hashCode(){ return 31 * name.hashCode() + age; }
}
```

WHY HASHCODE() MATTERS

HashMap/HashSet (Unit 5) store objects in buckets decided by hashCode(). If equals() says two objects are equal but their hashCodes differ, the map cannot find them. EQUAL objects MUST have equal hash codes — you override them together.

Constructor case	Syntax	Calling example
default (auto)	Compiler creates if none written	new Student()
parameterized	Student(int r, String n)	new Student(1, "Riya")

Constructor case	Syntax	Calling example
copy	Student(Student other)	new Student(s1)
chaining	this(...) as first statement	this(0, "unknown")
private (singleton)	private Student()	getInstance() (Unit 5 pattern)

OBJECTS & MEMORY IN 3 VARIABLES

obj reference (the name you use), ? hit memory: the object data on the HEAP, the static/class information loaded once in the METHOD AREA. new places the object on the heap; GC reclaims when no references point to it.

Exercise: *1) Write class Mobile with private brand, price, static count that increments on every new Mobile; print count. 2) Identify classes from "An ATM serves customers who insert a card, enter a PIN and withdraw cash" (write CRC cards). 3) Create a copy constructor for Mobile and show two objects are not the same identity. 4) Explain in 2 lines why balance should be private.*

UNIT 3 | INHERITANCE & RELATIONSHIPS

3.1 Inheritance — purpose & types

INHERITANCE

A mechanism where a NEW class (child/sub class) ACQUIRES the attributes and methods of an EXISTING class (parent/super class), then adds or overrides its own. Models an 'is-a' relationship. Java keyword: extends (single inheritance only; tree of classes).

PURPOSE OF INHERITANCE

1) REUSE: no need to rewrite common code. 2) EXTENSIBILITY: add new features by extending. 3) HIERARCHICAL organisation close to the real world (a Dog is an Animal). 4) Polymorphism base — a parent-typed variable can hold any child object. 5) Easier maintenance: fix behaviour once in the parent.

```
Hierarchy example  (single, multilevel, hierarchical):

              [Animal]          <- base (root)
               /    \
        [Bird]       [Mammal]   <- hierarchical (2 children)
         /  \            \
    [Sparrow][Penguin]   [Dog]  <- further levels  [multilevel]

Java:  class Mammal extends Animal {}         (1 parent only)
       class Dog  extends Mammal {}
Taxi is-a Vehicle   |   Manager is-a Employee   |   Book is-a Item

NOT allowed in Java:  class X extends A, B {}   (multiple via class)
                     -> use INTERFACES for multiple behaviour.
```

Type	Shape	Java support	Example
Single	one parent -> one child	YES (extends)	Dog extends Animal
Multilevel	A -> B -> C chain	YES	Animal -> Mammal -> Dog
Hierarchical	one parent, many children	YES	Animal -> Bird, Mammal, Fish
Multiple	child from 2+ classes	NO (class) / YES via interfaces	Duck implements Flyer, Swimmer
Hybrid (mix)	any combination	Only via interfaces	interface Foo + extends Bar

InheritanceDemo.java

```java
class Animal {
    String name;
    public Animal(String n){ name = n; }
    public void eat(){ System.out.println(name + " eats."); }
}
class Dog extends Animal { // Dog IS-A Animal
    public Dog(String n){ super(n); } // call parent constructor
    public void bark(){ System.out.println(name + " barks."); }
}
class Puppy extends Dog { // multilevel inheritance
    public Puppy(String n){ super(n); }
    @Override public void bark(){ System.out.println(name + " yaps."); }
}
public class InheritanceDemo {
    public static void main(String[] a){
        Puppy p = new Puppy("Tommy");
        p.eat(); // inherited from Animal
        p.bark(); // overridden behaviour
    }
}
```

3.2 The 'is-a' relationship and super

IS-A RULE

Use inheritance only when the child genuinely IS a kind of the parent. A Circle is-a Shape, a SavingsAccount is-a Account, a Rectangle IS-a Shape. If the relationship is more a 'has-a', use Association/Composition instead — a Car HAS-A Engine, it is NOT a kind of Engine.

IsARule.java

```
class Shape { public double area(){ return 0; } }
class Rectangle extends Shape { // Rectangle IS-A Shape OK
    double w, h;
    Rectangle(double w, double h){ this.w=w; this.h=h; }
    @Override public double area(){ return w * h; }
}
class Engine { void start(){ System.out.println("Engine running"); } }
class Car { // Car HAS-A Engine OK
    private Engine engine = new Engine(); // composition (not extends)
    void startCar(){ engine.start(); }
}
// Wrong: class Car extends Engine -> Car IS NOT an Engine. Avoid.
```

3.3 Association, Aggregation, Composition

ASSOCIATION

The weakest relationship: objects know each other and use each other (has-a, uses-a) but have INDEPENDENT lifetimes. Example: a Doctor and a Patient — a patient can exist without the doctor and vice versa. Implemented as a field/reference, method parameter, or local variable.

AGGREGATION (WEAK WHOLE-PART)

A special 'has-a': the whole CONTAINS parts, but parts can exist independently (empty relationship diamond). Example: a Department has Employees; an employee can exist after the department is closed. The part is passed IN from outside (constructor).

COMPOSITION (STRONG WHOLE-PART)

Strongest: the part cannot exist without the whole; the whole CREATES the parts (filled diamond). If the whole dies, parts die too. Example: a House has Rooms — remove the house, rooms are gone. Build parts inside the whole (new inside constructor).

```
UML relationship notation:
 1. Association   Doctor -----> Patient      (simple line, use arrow)
 2. Aggregation   Department <>----- Employee (hollow diamond at whole)
 3. Composition   House     <#>----- Room    (filled diamond)
 4. Inheritance   Dog       ----|> Animal    (hollow triangle)
 5. Dependency    Driver    ----> Car        (dashed arrow)

lifetimes:  Association: independent both sides
            Aggregation: parts independent (passed in)
            Composition: parts die with the whole (new inside)
```

Relationships.java

```
class Employee { String name; Employee(String n){ name=n; } }
class Department { // AGGREGATION (weak)
    private Employee[] staff;
    public Department(Employee[] e){ staff = e; } // employees come from OUTSIDE
    // employees live even if this Department object is gone
}
class Room { }
class House { // COMPOSITION (strong)
    private Room room = new Room(); // part CREATED inside whole
    // if house is garbage-collected, the room goes with it
}
```

3.4 Abstract Classes

ABSTRACT CLASS

A class that CANNOT be instantiated (no new AbstractX()). It may have abstract methods (signature only, no body) and also fully implemented methods/fields. Child classes MUST override the abstract methods (or be abstract too). Keyword: abstract.

```
ABSTRACT CLASS - what you can put inside:
+---------------------------------------------+
| abstract class Payment {                    |
|   private double amount;         // field   |
|   Payment(double a){...}         // constructor|
|   abstract void pay();           // abstract |
|   void printReceipt(){...}       // concrete |
|   static final double TAX = 0.18;// constant |
| }                                |          |
+---------------------------------------------+
```

ABSTRACT CLASSES VS INTERFACES (ONE-LINE VIEW)

Interface = a 100% abstract contract of capabilities (can implement many). Abstract class = a partially built base with shared code + some abstract methods (can extend only one). Since Java 8 interfaces can carry default & static methods too.

AbstractDemo.java

```java
abstract class Vehicle { // cannot be instantiated
    String reg;
    Vehicle(String r){ reg = r; }
    abstract void start(); // no body - must override
    void fuelUp(){ System.out.println("fuelled " + reg); } // concrete method
}
class Bike extends Vehicle {
    Bike(String r){ super(r); }
    @Override void start(){ System.out.println("Bike " + reg + " kicked"); }
}
public class AbstractDemo {
    public static void main(String[] a){
        Bike b = new Bike("MH-12-3456");
        b.start(); b.fuelUp();
        // Vehicle v = new Vehicle("x"); -> COMPILE error (abstract)
        Vehicle v = new Bike("MH-01-9999"); // up-casting: OK
        v.start(); // runs Bike.start() (polymorphism)
    }
}
```

3.5 Interfaces

INTERFACE

A pure CONTRACT that lists method signatures (and constants) that implementing classes MUST provide. A class can implement MULTIPLE interfaces — this is how Java achieves 'multiple-inheritance-like' behaviour. Java 8: default methods give optional default bodies; static methods allowed. Keyword: implements.

InterfaceDemo.java

```java
interface Flyer { void fly(); } // contract of "things that fly"
interface Hunter { void hunt(); }
class Bat implements Flyer, Hunter { // multiple capability - NOT multiple class
    public void fly(){ System.out.println("Bat flutters"); }
    public void hunt(){ System.out.println("Bat catches insects"); }
}
public class InterfaceDemo {
    public static void main(String[] a){
        Flyer f = new Bat(); // interface-typed reference
        Hunter h = new Bat();
        f.fly();
        h.hunt();
    }
}
// java.util.List, java.util.Map, Comparable, Runnable, Comparator are
// interfaces you meet every day.
```

Point	Abstract class	Interface
Members	fields + concrete + abstract methods	constants + abstract methods (+ default/static since 8)

Point	Abstract class	Interface
Instantiation	Not allowed	Not allowed
Inheritance	extends (ONE)	implements (MANY)
Constructor	Yes	No
Use when	classes share implementation (is-a with code)	classes share capability (can-do contract)
Java examples	AbstractList, HttpServlet	List, Map, Comparable, Runnable

KEY POINT

Real world: Comparable<E> lets Collections.sort() work on YOUR class (a contract that libraries rely on). Runnable / Callable are the threading contract. Spring uses interfaces for dependency injection — code programs against contracts, not concrete classes (design that keeps large apps testable).

3.6 Constructors, super and final in inheritance

CONSTRUCTOR RULE IN INHERITANCE

A child constructor MUST call a parent constructor (implicitly super() if you write none). super(...) must be the FIRST statement. If the parent has only a parameterized constructor, the child MUST call it explicitly. Objects are built top-down (parent part first).

SuperRule.java

```java
class Employee {
    String name;
    Employee(String n){ name = n; } // only parameterized constructor
}
class Manager extends Employee {
    String team;
    Manager(String n, String t){
        super(n); // must be first statement!
        team = t;
    }
}
// ERROR: class Manager extends Employee { Manager(){ } }
// -> implicit super() fails: Employee has no no-arg constructor
```

FINAL KEYWORD — STOPS THE CHAIN

final class : cannot be inherited (String is final). final method : cannot be overridden. final field : constant (must be assigned once). These protect behaviour critical to security and design — String is final so its behaviour can never be hijacked.

FinalDemo.java

```java
final class Safety { public void doIt(){ } } // cannot be extended
class Child { final void lock(){ } } // cannot be overridden
class Grand extends Child { /* @Override void lock() -> ERROR */ }
class Config { public static final int MAX = 100; } // compile-time constant
```

Relationship	Meaning	Lifetime of parts	Strong link	Real example
Association	objects use each other (uses-a)	independent	-	Doctor -> Patient
Aggregation	whole has parts (has-a, weak)	parts outlive whole	hollow <>,	Department has Employees
Composition	whole OWNS parts (has-a, strong)	parts die with whole	filled <>,	House has Rooms
Inheritance	child IS-A kind of parent	object is one identity	triangle |>,	Dog is-an Animal

IS-A VS HAS-A MEMORY TRICK

Show the sentence in English: "A Bike IS-A Vehicle" → inheritance. "A Bike HAS-A Wheel" → composition field. "A Clinic HAS-A Doctor" (working with) → aggregation or association. If you are unsure, prefer COMPOSITION over inheritance (effective-Java rule).

Exercise: *1) Draw an inheritance tree for Vehicle -> Car/Bike -> ElectricCar and annotate each edge is-a. 2) Model "University has Departments, Department has Professors" - say which is aggregation vs composition and write the classes. 3) Define interface Printable with print(); make Invoice and Report implement it. 4) Fill the access-modifier table for protected in your own words.*

UNIT 4 | POLYMORPHISM

4.1 Introduction to Polymorphism

DEFINITION: POLYMORPHISM

From Greek poly = many, morph = forms. The SAME method name (or operator) means different things depending on the situation. Two major kinds in Java: 1) COMPILE-TIME (static) → method OVERLOADING, resolved by the compiler. 2) RUN-TIME (dynamic) → method OVERRIDING, resolved when the program runs via dynamic method dispatch.

```
"Same name, many forms"
int x = 5, y = 7;          " + " on ints   -> Adds numbers  (5+7=12)
String s = "A" + "B";      " + " on String -> concatenates text (AB)
same operator +, two different meanings  =  ad-hoc polymorphism

print(12)          print(3.14)       print("Hi")
-> System.out.print is overloaded for every primitive type and Object.

Shape s = new Circle();   s.area()   -> Circle-area (run time)
Shape s = new Square();   s.area()   -> Square-area (run time)
the call s.area() is decided at RUN TIME depending on the object.
```

KEY POINT

Farida's rule of thumb: OVERLOADING = same class, different parameters, decided at compile time. OVERRIDING = different classes (parent/child), same signature, decided at run time.

4.2 Method Overloading (compile-time / static polymorphism)

OVERLOADING

Declaring SEVERAL methods with the SAME NAME in the same class but DIFFERENT parameter lists (different count OR different types OR different order). The compiler picks the correct version from the arguments at compile time. Return type alone cannot distinguish overloads.

Calculator.java

```java
public class Calculator {
    // SAME name "add", 4 different parameter lists = overloads
    int add(int a, int b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
    double add(double a, double b) { return a + b; }
    String add(String a, String b) { return a.concat(b); }
    public static void main(String[] a){
        Calculator c = new Calculator();
        System.out.println(c.add(2, 3)); // int,int -> 5
        System.out.println(c.add(2, 3, 4)); // 3 params -> 9
        System.out.println(c.add(2.5, 1.5)); // double -> 4.0
        System.out.println(c.add("Ja","va")); // String -> Java
    }
}
```

RULES OF OVERLOADING

Allowed changes: number of parameters, types of parameters, order of types. NOT allowed: overload by return type alone (compile error), and you cannot re-declare the exact same signature twice. Note: overloads exist since C and are totally different from overriding.

OverloadRules.java

```
// Deciding "z = f(1, 2)" when both f(int,int) and f(double,double) exist:
// -> compiler picks the MOST SPECIFIC match: f(int,int)
// Valid: m(int, double) and m(double, int) (different order)
// Invalid: int m(int x) and void m(int x) (same signature)
// Constructors are overloaded too: Box(), Box(int w), Box(int w,int h)
```

4.3 Method Overriding (run-time / dynamic polymorphism)

OVERRIDING

A child class provides its OWN implementation of a method that is defined in its parent, keeping the SAME name, SAME parameter list and SAME return type (covariant allowed). The parent method is overridden for objects of the child class. Must be marked @Override (optional but recommended).

DYNAMIC METHOD DISPATCH

When we call an OVERRIDDEN method through a PARENT-TYPED reference, the JVM looks at the ACTUAL object type at run time and calls the child version. This is the engine of run-time polymorphism.

OverrideDemo.java

```java
class Payment {
    public void pay(double amount){
        System.out.println("Generic payment of Rs " + amount);
    }
}
class CashPayment extends Payment {
    @Override public void pay(double amount){ // same signature
        System.out.println("Cash paid Rs " + amount);
    }
}
class CardPayment extends Payment {
    @Override public void pay(double amount){
        System.out.println("Card charged Rs " + amount + " (2% fee)");
    }
}
public class OverrideDemo {
    public static void main(String[] a){
        Payment p; // parent-typed variable
        p = new CashPayment(); p.pay(100); // runs CASH version
        p = new CardPayment(); p.pay(100); // runs CARD version
        // same line of code, different behaviour = runtime polymorphism
    }
}
```

RULES OF OVERRIDING

Same signature & compatible return type; access cannot REDUCE (public can override protected, not the reverse); cannot override static, private or final methods; constructors are never overridden; checked exceptions cannot be widened. Use super.method() to call the parent version.

OverrideRules.java

```java
class Parent {
    protected void show(){ System.out.println("Parent"); }
}
class Child extends Parent {
    @Override public void show(){ // widen access: OK
        super.show(); // call parent too
        System.out.println("Child");
    }
}
// ERROR if you try: private void show() (narrowing)
```

4.4 Static vs Run-time Polymorphism — comparison

Point	Static (overloading)	Run-time (overriding)
Known at	COMPILE time	RUN time
Same class?	Yes (same class, many versions)	No (parent child classes)
Signature	Different parameter lists	Same signature
Keyword	none (overloads)	@Override
Bond	Compile-time binding (early)	Dynamic dispatch (late binding)
Speed	Faster (resolved by compiler)	Small overhead (vtable look-up)
Example	System.out.println(...) many versions	Shape.area() -> Circle/Square versions

PolyDemo.java

```java
public class PolyDemo {
    static class Media { void play(){ System.out.println("playing media"); } }
    static class Video extends Media { @Override void play(){ System.out.println("video playing"); } }
    static class Audio extends Media { @Override void play(){ System.out.println("audio playing"); } }
    public static void main(String[] a){
        Media[] list = { new Video(), new Audio(), new Media() };
        for (Media m : list) m.play(); // the SAME loop runs 3 behaviours
    }
}
```

KEY POINT

Real world of polymorphism: every UI framework (paint() in Swing/Android View), Hibernate + Spring proxies (runtime proxies override methods to add transactions), Stream pipelines (functions as objects), strategy patterns, and Java's own toString()/equals() called polymorphically everywhere. Excel-like apps let a Shape list redraw itself automatically — that is polymorphism.

4.5 instanceof and safe down-casting

CastingDemo.java

```java
public class CastingDemo {
    static class Animal { void sound(){} }
    static class Dog extends Animal { void bark(){ System.out.println("Woof"); } }
    public static void main(String[] a){
        Animal a1 = new Dog(); // up-cast (implicit, safe)
        if (a1 instanceof Dog) { // run-time type test
            Dog d = (Dog) a1; // down-cast (explicit)
            d.bark();
        }
        // Since Java 16: if (a1 instanceof Dog d) { d.bark(); } (pattern matching)
    }
}
```

4.6 Polymorphism in Java vs C++ (syllabus comparison)

VIRTUAL KEYWORD (C++)

In C++, a method is polymorphic ONLY if marked virtual; in Java every non-static non-final method is virtual automatically. That is the main reason Java reads easier for OO examples — you never forget the keyword.

Feature	C++	Java
Polymorphic method mark	must write virtual	automatic (non-static, non-final)
Multiple inheritance	class A : B, C allowed	only via interfaces
Operator overloading	supported (a+b means custom)	NOT supported (+)
Destructor	~Class() explicit	GC (no destructor)
Overload by return type	sometimes ambiguous	compile error

StrategyDemo.java

```java
// The STRATEGY design pattern = polymorphism in action:
interface SortStrategy { void sort(int[] a); } // contract
class QuickStrategy implements SortStrategy {
    public void sort(int[] a){ /* quick sort ... */ }
}
class BubbleStrategy implements SortStrategy {
    public void sort(int[] a){ /* bubble sort ... */ }
}
class DataProcessor {
    private SortStrategy s; // "plug in" behaviour
    public void setStrategy(SortStrategy s){ this.s = s; }
    public void process(int[] data){ s.sort(data); } // same code, many behaviours
}
// caller decides at RUNTIME: processor.setStrategy(new QuickStrategy());
```

KEY POINT

Every modern framework relies on this: your override of paint() in Android, doGet()/doPost() in servlets, execute() in Selenium, listeners in Swing — the framework calls YOUR method through a parent reference at run time.

Exercise: *1) Create Shape subclasses Circle and Square each overriding area(); build an array of Shape and print all areas (night-build the PolyDemo pattern). 2) Overload area() to also accept a circle radius. 3) Try to overload a method by return type only and note the compiler error. 4) Answer: why is overriding called "dynamic" and overloading "static"?*

UNIT 5 | STRINGS, EXCEPTIONS, THREADS, COLLECTIONS & CASE STUDIES

5.1 Strings in Java

STRING FACTS

String objects are IMMUTABLE (cannot be changed once made) and stored in the String pool. String vs new String("x") differ (pooling). String = series of Unicode chars. StringBuilder/StringBuffer are MUTABLE; use Builder inside single threads (faster), Buffer in threads.

StringsDemo.java

```java
public class StringsDemo {
    public static void main(String[] a){
        String s = "Hello"; // pooled literal
        String t = "Hello"; // same pool entry, s == t
        String u = new String("Hello"); // new object, s != u
        System.out.println((s == t) + " " + (s == u) + " " + s.equals(u));
        System.out.println(s.length()); // 5
        System.out.println(s.toUpperCase()+" "+s.charAt(1)); // HELLO e
        System.out.println("a,b,c".split(",").length); // 3
        System.out.println("hello".substring(1, 3)); // el
        // IMMUTABLE: every operation returns a NEW string
        StringBuilder sb = new StringBuilder("Hi");
        sb.append(" there").insert(0, ">> ").reverse();
        System.out.println(sb); // "ereht iH >>" (mutated in place)
    }
}
```

KEY POINT

Real world: logs are mainly string building; JSON/XML serializers (Jackson/Gson) build big strings with builders; password hashing and URL encoding normalise strings; String.intern() powers canonicalisation.

5.2 Exception Handling

WHY EXCEPTIONS

Runtime errors (div by zero, file missing, null pointer, bad cast) would kill the program. Java lets you THROW an exception object and CATCH it, so the program can recover instead of crashing.

```
Exception hierarchy (Throwable):
     Throwable
       |-- Error        (OutOfMemory, StackOverflow - JVM; do NOT catch)
       |-- Exception
             |-- RuntimeException  (Arithmetic, NullPointer, IndexOutOfBounds)
             |-- checked exceptions: IOException, SQLException, FileNotFound
try  { risky code }  catch(Ex e){ handle }  finally{ always runs }
throws = method declares it may pass the error up
throw  = you deliberately raise an error
```

ExceptionsDemo.java

```java
import java.util.Scanner;
public class ExceptionsDemo {
    static int divide(int a, int b) throws ArithmeticException {
        if (b == 0) throw new ArithmeticException("division by zero");
        return a / b;
    }
    public static void main(String[] a){
        Scanner sc = new Scanner(System.in);
        try {
            int x = sc.nextInt();
            int y = sc.nextInt();
            System.out.println(divide(x, y));
        } catch (ArithmeticException e) {
            System.out.println("caught: " + e.getMessage());
        } finally {
            System.out.println("finally always runs (cleanup here)");
        }
        // multi-catch: catch (IOException | SQLException e)
    }
}
```

MyException.java

```java
class MyException extends Exception { // custom (checked) exception
    MyException(String m){ super(m); }
}
class Account {
    private double bal = 500;
    void withdraw(double amt) throws MyException {
        if (amt > bal) throw new MyException("insufficient balance");
        bal -= amt;
    }
    public static void main(String[] a) {
        try { new Account().withdraw(900); }
        catch (MyException e){ System.out.println(e.getMessage()); }
    }
}
```

KEY POINT

Best practice: catch only what you can handle, prefer custom exceptions with clear messages for business rules, use finally or try-with-resources to close files/sockets. DO NOT swallow exceptions silently. Real world: every web framework maps exceptions to HTTP error codes (500/404) through a global handler.

5.3 Multi-threading

THREADS

A thread is an independent path of execution inside one program (process). Multi-threading lets a program do several jobs at once (download + play + UI). Two ways to create: implement Runnable (preferred, Java class can only extend one class) or extend Thread. The scheduler shares CPU time-slices.

ThreadingDemo.java

```java
public class ThreadingDemo {
    public static void main(String[] a) throws InterruptedException {
        Runnable task = () -> { // Lambda implementing run()
            for (int i = 1; i <= 5; i++)
                System.out.println(Thread.currentThread().getName() + ":" + i);
        };
        Thread t1 = new Thread(task, "printer-1");
        Thread t2 = new Thread(task, "printer-2");
        t1.start(); t2.start(); // start scheduling now
        t1.join(); t2.join(); // main waits for both
        System.out.println("all threads done");
    }
}
```

SYNCHRONIZATION

When threads SHARE data (e.g., a bank balance), racing writes corrupt it. synchronized on a method/block locks the object so only one thread enters at a time — the classic "accountwithdrawal must be atomic" rule (see ATM case study).

SynchronizedDemo.java

```java
class BankBalance {
    private int balance = 1000;
    public synchronized void withdraw(int amt){ // LOCK = this object
        if (balance >= amt) balance -= amt; // safe critical section
        else System.out.println("denied");
    }
}
// Without synchronized, two threads could both read balance=1000 and
// both deduct - leaving the account wrong. synchronized serialises them.
```

KEY POINT

Real world: web servers (Tomcat) run thousands of threads; every request = a thread. Executors.newFixedThreadPool(10) is the modern way (Executors framework) instead of raw new Thread. Volatile, AtomicInteger and concurrent collections (ConcurrentHashMap) complete the toolkit.

5.4 Data Collections (java.util)

COLLECTIONS FRAMEWORK

Ready-made containers for groups of objects: List (ordered, indexed, duplicates), Set (no duplicates), Map (key-value). Every framework class is built with the OO concepts of the earlier units (interface families + generics ■ + polymorphism).

```
      Collection<E> (interface)                    Map<K,V> (interface)
          |              \                              |-- HashMap
         List             Set                           |-- TreeMap (sorted)
    |-- ArrayList    |-- HashSet      keys unique; key -> value
    |-- LinkedList   |-- LinkedHashSet
    |-- Vector       |-- TreeSet (sorted)
```

CollectionsDemo.java

```java
import java.util.*;
public class CollectionsDemo {
    public static void main(String[] a){
        // LIST - ordered, keeps duplicates, random access
        List<String> names = new ArrayList<>(List.of("Riya","Aman","Riya"));
        names.add("Sara"); names.remove(0);
        System.out.println(names); // [Aman, Riya, Sara]
        // SET - removes duplicates
        Set<Integer> nums = new TreeSet<>(List.of(5,2,8,2,5));
        System.out.println(nums); // [2, 5, 8] (sorted, unique)
        // MAP - key -> value (HashSet/Map use hashCode + equals from Unit 2!)
        Map<String, Integer> marks = new HashMap<>();
        marks.put("Riya", 92); marks.put("Aman", 88);
        System.out.println(marks.get("Riya") + " " + marks.size());
        // generics + lambda: polymorphic iteration
        names.stream().filter(s -> s.startsWith("R")).forEach(System.out::print);
    }
}
```

Interface	Implementation	Ordered?	Duplicates?	Use for
List	ArrayList / LinkedList	Yes (indexes)	Yes	items list, history
Set	HashSet / TreeSet	No / sorted	No	unique ids, tags
Map	HashMap / TreeMap	No / sorted	Unique keys	lookup tables, config
Queue	ArrayDeque / PriorityQueue	Yes	Yes	tasks, scheduling

5.5 Case Study — ATM Machine

```
Classes from the ATM problem statement (a customer withdraws cash
with a valid card and pin):
[Card]--->[ATM]<--->[Bank]  and  [Bank] owns [Account] (composition)

hidden design decisions you will reuse:
* balance is private (Unit 2 encapsulation)
* withdraw() is synchronized (Unit 5 concurrency)
* ATM is an interface family (Unit 3): CardReader, Dispenser
* runtime polymorphism picks the right bank (Unit 4)
```

ATM.java

```java
class ATM {
    private double cashAvailable = 100000;
    public void withdraw(Card c, int pin, double amt) throws Exception {
        if (!c.verify(pin)) throw new Exception("Invalid PIN"); // validation
        synchronized (this) { // one withdrawal at a time
            if (amt > cashAvailable) throw new Exception("Machine out of cash");
            Bank b = c.getBank();
            b.debit(c.getAccountNo(), amt); // message passing to Bank
            cashAvailable -= amt; // dispense notes
        }
    }
}
class Bank {
    private Map<String, Account> accounts = new HashMap<>(); // collections
    public synchronized void debit(String acc, double amt) throws Exception{
        Account a = accounts.get(acc); // lookup via Map
        a.withdraw(amt); // reuses Account.withdraw
    }
}
class Account {
    private double balance; // encapsulation
    void withdraw(double amt) throws Exception {
        if (amt > balance) throw new Exception("Insufficient balance");
        balance -= amt;
    }
}
```

5.6 Case Study — Library Management System

```
UML-ish class picture (is-a, has-a, uses-a):
      Item (abstract)  <|--- Book, DVD          (inheritance)
      Member  o---> Loan records                (association)
      Library #---> Book copies                 (composition)
      Fine is calculated by a method loan.calculateFine()
```

Library.java

```java
import java.util.*;
abstract class Item { // Unit 3: abstract class
    private String title; private String id;
    Item(String t, String i){ title=t; id=i; }
    abstract double calculateLateFine(int days);
    public String getTitle(){ return title; }
}
class Book extends Item {
    private String author;
    Book(String t, String i, String a){ super(t, i); author = a; }
    @Override double calculateLateFine(int days){ return 5.0 * days; } // Rs5/day
}
class DVD extends Item {
    DVD(String t, String i){ super(t, i); }
    @Override double calculateLateFine(int days){ return 20.0 * days; }
}
public class Library {
    private Map<String, Item> items = new HashMap<>(); // collection
    private Set<String> issued = new HashSet<>();
    public void addItem(Item it){ items.put(id(it), it); }
    private String id(Item it){ return it.getTitle(); }
    public void issue(String title){
        if (!issued.add(title)) throw new RuntimeException("already issued");
        System.out.println("issued " + title);
    }
    public static void main(String[] a){
        Library lib = new Library();
        lib.addItem(new Book("DSA", "B1", "Lafore"));
        lib.addItem(new DVD("Tutorials", "D1"));
        lib.issue("DSA"); // polymorphism: both
        // the same array could be scanned with Item references.
    }
}
```

Unit 1 Modelling real entities as classes (Member, Book).	**Unit 2** private fields, constructors, encapsulation of fine).	**Unit 3** Item abstract; Book/DVD extend; Library HAS-A books.	**Unit 4** one Item reference calls overriding calculateLateFine.
Unit 5 Map/Set collections + custom RuntimeException.	**Improve 1** Add Member borrow list with due dates.	**Improve 2** Persist library to a file (Unit 1 IO).	**Improve 3** Use threads for concurrent borrowers (Unit 5).

5.7 Generics, Executors & the OOP pattern book

GENERICS (PARAMETERISED TYPES)

Generic classes/methods work for any type: ArrayList<String>, HashMap<String,Integer>. The type is a parameter checked at COMPILE time — no casts, no ClassCastException.

ModernTools.java

```java
import java.util.concurrent.*;
public class ModernTools {
    // GENERIC method: works on any Comparable type
    static <T extends Comparable<T>> T max(T a, T b){
        return a.compareTo(b) >= 0 ? a : b;
    }
    public static void main(String[] a) throws Exception {
        System.out.println(max(3, 9)); // 9
        System.out.println(max("apple", "mango")); // mango
        // ExecutorService = thread-pool (do not new Thread manually)
        ExecutorService pool = Executors.newFixedThreadPool(4);
        for (int i = 1; i <= 6; i++){
            final int job = i;
            pool.submit(() -> System.out.println("job " + job +
                " on " + Thread.currentThread().getName()));
        }
        pool.shutdown();
    }
}
```

Point	String	StringBuilder / StringBuffer
Mutability	immutable (new object per change)	mutable (changes in place)

Point	String	StringBuilder / StringBuffer
Thread safe	yes (safe by default)	Buffer=yes, Builder=no
Concatenation in loop	slow (O(n^2) copies)	fast (amortised O(n))
Memory	String pool + constant overhead	single growable char array
Use for	fixed text, keys in maps	building logs, SQL, JSON

KEY POINT

Design patterns you have now met in this course: Singleton (private constructor, Unit 2), Template Method (abstract class + hooks, Unit 3), Strategy (interface + runtime switch, Unit 4), Factory (static method creating objects), Observable (listener callbacks = polymorphism). Patterns are simply advanced OO structure.

Exercises: *1) Build a mini ATM menu (deposit/withdraw/balance) with Scanner and catch bad amounts. 2) Make Library store members in a Map and print sorted names. 3) Write a producer/consumer with threads sharing one queue. 4) Explain with one example how HashMap uses equals/hashCode (trace Riya 92).*